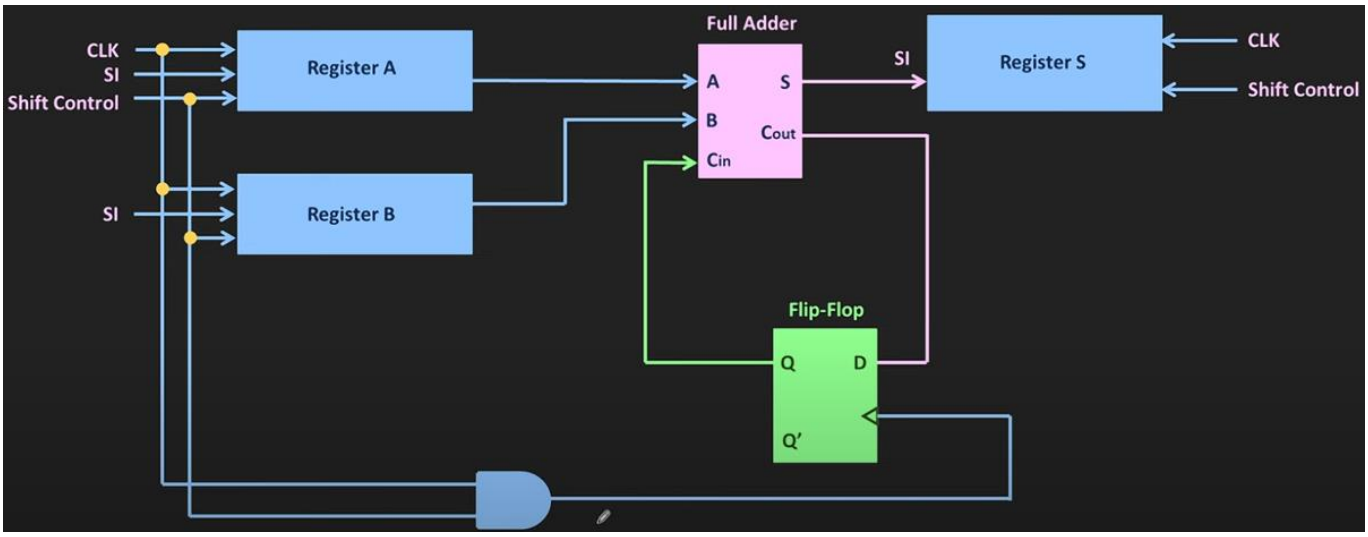
LAB 5: Serial Adder

Date:30 May of 2025



Proffesor: Dr. Francisco Javier Rodríguez Navarrete

Students: Cesar Eduardo Inda Ceniceros

Alonso Emmanuel López Macías

Team: 14

Introduction

In this lab, a 4-bit serial adder was developed using the Verilog Hardware Description Language (HDL), with the objective of strengthening the design of sequential systems and the modular integration of subcomponents. Unlike parallel adders, the serial adder operates bit by bit on each clock cycle, using shift registers and control logic to optimize hardware resource usage.

During development, it was necessary to implement an architecture composed of multiple submodules, including input and output registers, a 1-bit full adder, carry logic, and a finite state machine (FSM) to coordinate the addition process. This lab posed a greater challenge compared to previous exercises, as it required a deeper understanding of sequential behavior and module interaction.

Understanding the logic behind a serial adder and being able to implement it in Verilog is essential for any digital system designer, as it enables the creation of more efficient circuits in terms of area and power consumption. Verilog, as an industry-standard hardware language, provides a robust platform for modeling and simulating complex digital behaviors. Mastering its use not only facilitates FPGA implementation but also opens the door to designing custom solutions in embedded devices and high-performance computing systems.

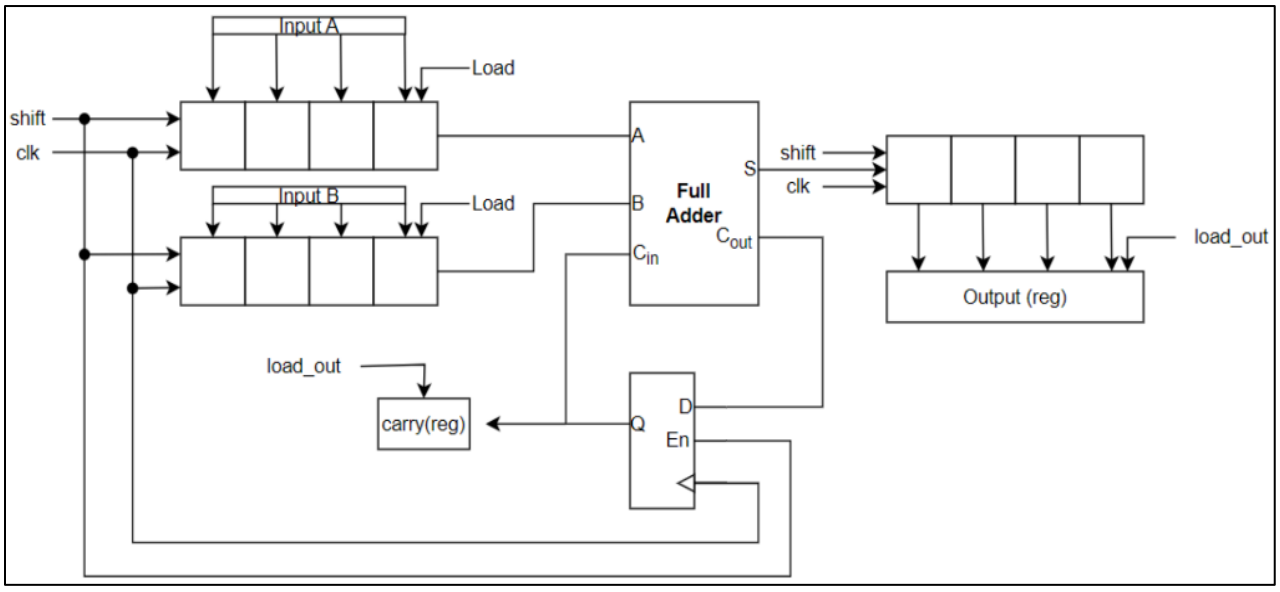


Figure: 1 Serial Adder Diagram

Requirements

- The project consists of designing a **4-bit serial adder** with specific inputs and outputs. The inputs include a ***clock (clk),*** synchronous ***reset (rst),*** a start signal to initiate the addition (start), and two ***4-bit inputs (in_a and in_b)*** representing the numbers to be added.
- The outputs are a ***4-bit result (sum) and a carry-out (c).***
- The operation involves capturing the input values when start is low, and beginning the addition process when start transitions from low to high.
- The addition is performed bit by bit using shift registers. Once completed, the result is stored in an output register until a new operation occurs. The reset signal restarts the process when activated.

The proposed microarchitecture for the 4-bit serial adder includes several submodules for both control and data flow. Since internal signals like shift and ***load_out*** are not external inputs, a control circuit must be designed to generate them and manage the addition sequence. This architecture ensures organized sequential processing and modular design.

Control Logic:

- ***control_fsm:*** A finite state machine that starts the addition and generates control signals **(shift, load_out).**
- counter: Tracks the number of bits processed during the addition.

Data Flow Modules:

- ***input_reg:*** A shift registers with enable and parallel load to receive in_a and in_b when start is low and to shift bits during the operation.
- ***full_adder:*** A 1-bit full adder that performs the core bitwise addition.
- ***output_reg:*** A shift registers that stores and shifts the result bits.
- ***carry_logic***: A register to store and forward the carry between additions.
- ***output registers:*** Hold the final result until a new operation is performed.

Note: *In the data flow and control logic diagrams, the signals may have different names. For example, the 'busy' signal in the control logic corresponds to the 'shift' signal in the data flow.*

Development

For all considerations was better use **a Visual Studio Code Extension** for see easier the sintaxis in the highlight's words, because Quartus use a black text in so much context and this is some difficulty for write a big text or descriptions. In this we use this extension:

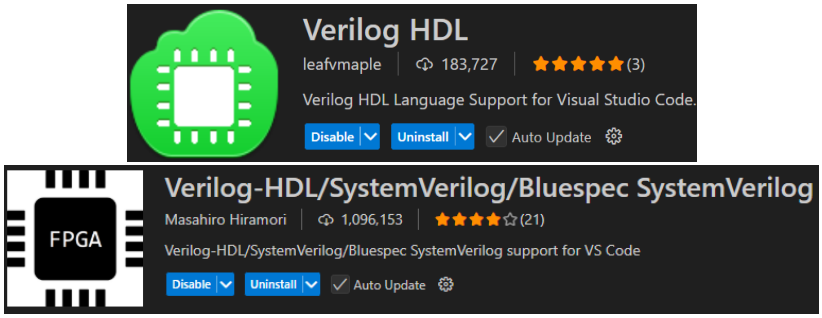


Figure: 2 Verilog Visual Studio Extension

The first consideration in this practice consisted of a full adder to be integrated into the 4 bits of each input value, in this case "A" and "B".

```
/*This file represents a 1-bit adder with carry-in and carry-out,
    so that it can be used in other sections of our design as an instance.*/

module full_adder (
    input wire a,
    input wire b,
    input wire cin,
    output wire sum,
    output wire cout
);
    assign sum = a ^ b ^ cin;
    assign cout = (a & b) | (a & cin) | (b & cin);


endmodule
```

Figure: 3 full adder. V

The second consideration was to take into account the use of carry logic; in this case, the module will store the "carry" (carry register between cycles).

```
module carry_logic (
    input wire clk,
    input wire rst,
    input wire load,
    input wire carry_in,
    output reg carry_out
);
    always @(posedge clk or posedge rst) begin
        if (rst)
            carry_out <= 0;
        else if (load)
            carry_out <= carry_in;
    end
endmodule
```

Figure: 4 Carry_logic. V

The other integrations was the implementation of input registers this for the shifter registers and parallel load in the enable. Is used for **“in_a”** and **in_b”.**

```
module input_reg (
    input wire clk,
    input wire rst,
    input wire load,
    input wire shift,
    input wire [3:0] data_in,
    output wire lsb_out
);
    reg [3:0] data;

    always @(posedge clk or posedge rst) begin
        if (rst)
            data <= 4'd0;
        else if (load)
            data <= data_in;
        else if (shift)
            data <= {1'b0, data[3:1]}; // desplaza a la derecha
    end

    assign lsb_out = data[0];
endmodule
```

Figure: 5 input_reg. v

In this file, the state machine will be initiated. When the sum == "start" goes high, it generates the shift and load_out signals, and activates "done" when it finishes. For this part only we are going to put a brief from the code because are around 90 lines in the code but the critical section is as following:

```
module control_fsm (
    input wire clk,
    input wire rst,
    input wire start,
    input wire count_done,
    output reg load_inputs,
    output reg load_out,
    output reg shift,
    output reg count_enable,
    output reg done
);
```

```
typedef enum reg [1:0] {
    IDLE  = 2'b00,
    LOAD  = 2'b01,
    SHIFT = 2'b10,
    DONE  = 2'b11
} state_t;

state_t current_state, next_state;
```

Figure: 6 fsm_part1.v

The transitions states are as following:

```
always @(*) begin
    case (current_state)
        IDLE:
            next_state = (start) ? LOAD : IDLE;
        LOAD:
            next_state = SHIFT;
        SHIFT:
            next_state = (count_done) ? DONE : SHIFT;
        DONE:
            next_state = (!start) ? IDLE : DONE;
        default:
            next_state = IDLE;
    endcase
end
```

Figure: 7 fsm_transictions.v.

And the register zone while “start” is == true || 1

```
always @(posedge clk or posedge rst) begin
    if (rst)
        done <= 0;
    else if (current_state == DONE && start) // activo mientras mantienes presionado start
        done <= 1;
    else
        done <= 0;
end


endmodule
```

Figure: 8 always for use "start”. v

The other critical section is the “counter” file because this is an indicator when 4 bits were added.

```
module counter (
    input wire clk,
    input wire rst,
    input wire enable,
    output reg done
);

    reg [2:0] count;

    always @(posedge clk or posedge rst) begin
        if (rst)
            count <= 0;
        else if (enable) begin
            if (count == 3)
                count <= 0;
            else
                count <= count + 1;
        end
    end

    always @(*) begin
        done = (count == 3);
    end

endmodule
```

Figure: 9 counter serial-adder. V

The file that, in this case, would be conditioned by the complete logic of the project is the ***"serial_adder_top".*** This file calls the internal signals from other files: the state machine, the counter, the registers, and each of the partial sums along with the carry logic and final values.

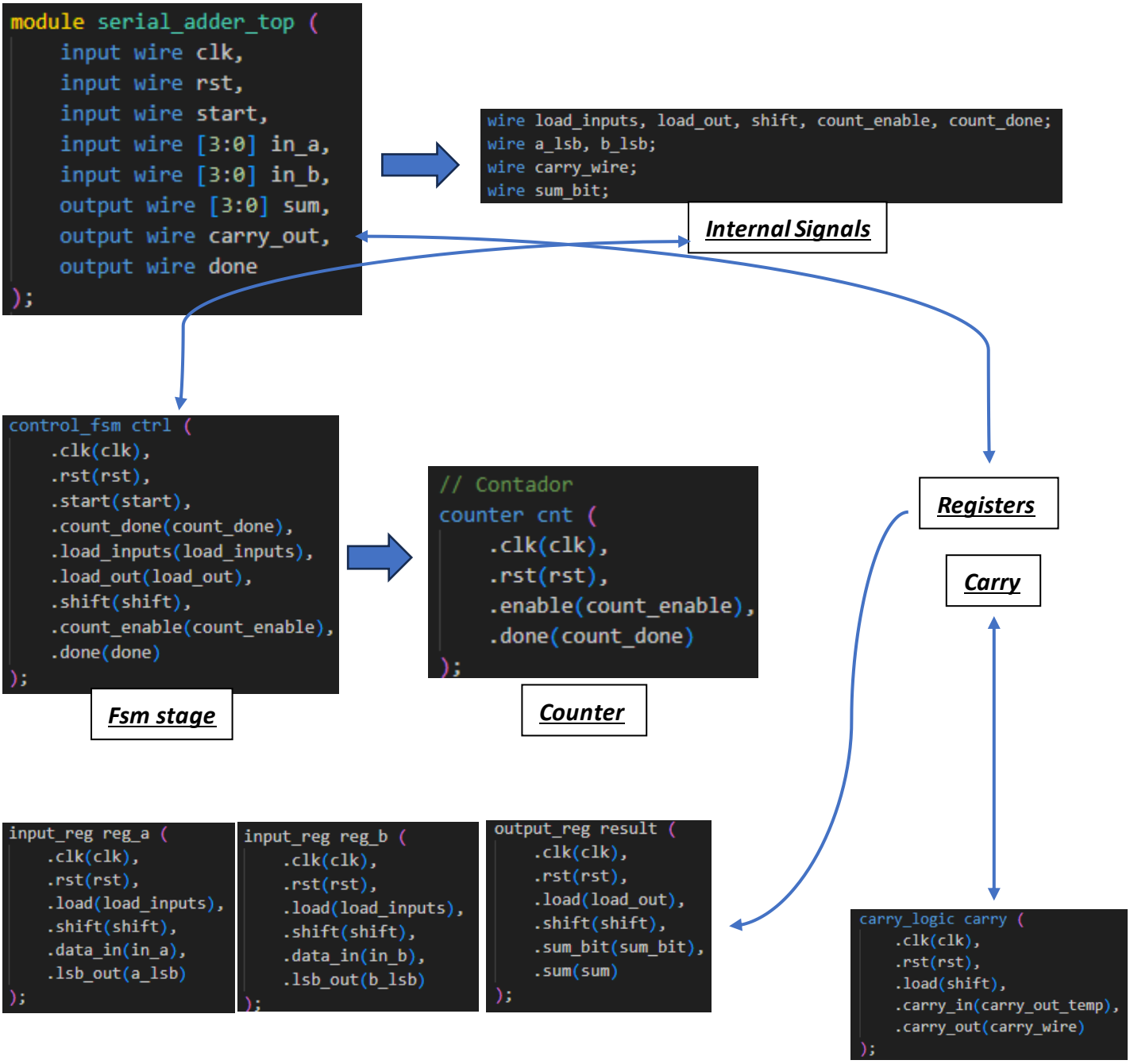


Figure: 10 serial_adder.v. v

Testbench

For the testbench implementation we took the next considerations because we need to add a "**DUT" (Device Under Test)"** section, and for use the monitor display representation and see the operations that were completed in the correct form.

```
`timescale 1ns / 1ps

module tb_serial_adder;

    reg clk;
    reg rst;
    reg start;
    reg [3:0] in_a, in_b;
    wire [3:0] sum;
    wire carry_out;
    wire done;

    // DUT
    serial_adder_top uut (
        .clk(clk),
        .rst(rst),
        .start(start),
        .in_a(in_a),
        .in_b(in_b),
        .sum(sum),
        .carry_out(carry_out),
        .done(done)
    );

    // Clock: 10ns period (100MHz)
    always #5 clk = ~clk;

    initial begin
        $display("Inicio de testbench para el serial adder");
        $monitor("T=%0t | A=%b, B=%b | SUM=%b | CARRY=%b | DONE=%b", $time, in_a, in_b, sum, carry_out, done);

        // Inicialización
        clk = 0;
        rst = 1;
        start = 0;
        in_a = 0;
        in_b = 0;

        #20;
        rst = 0;
```

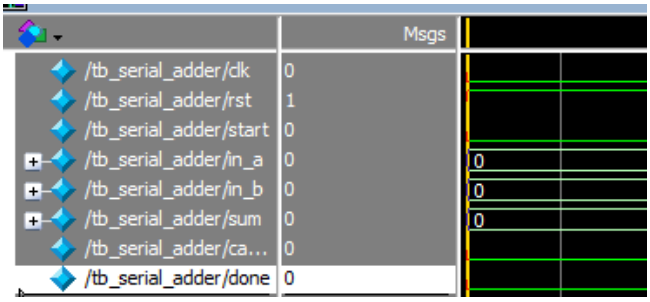
Figure: 11 Testbench_Part1.v

We used a cycle for see a test1, test2 and continues test for verify the correct result and the correct operation function.

```
        // Primera prueba: 3 + 5 = 8
        in_a = 4'b0011;
        in_b = 4'b0101;
        start = 1;
        #10;
        start = 0;

        wait(done);
        #10;

        // Segunda prueba: 7 + 9 = 16 (carry_out = 1)
        rst = 1; #10; rst = 0;
        in_a = 4'b0111;
        in_b = 4'b1001;
        start = 1; #10; start = 0;
        wait(done);
        #10;

        // Tercera prueba: 0 + 0 = 0
        rst = 1; #10; rst = 0;
        in_a = 4'b0000;
        in_b = 4'b0000;
        start = 1; #10; start = 0;
        wait(done);
        #10;

        // Fin de simulación
        $display("Fin de pruebas.");
        $stop;
    end
endmodule
```

Figure: 12 Testbench_Part2.v

Results are the next in ModelSim- Altera:




```
# Inicio de testbench para elserial adder
# T=0 | A=0000, B=0000 | SUM=0000 | CARRY=0 | DONE=0
# T=20000 | A=0011, B=0101 | SUM=0000 | CARRY=0 | DONE=0
# T=45000 | A=0011, B=0101 | SUM=0000 | CARRY=1 | DONE=0
# T=75000 | A=0011, B=0101 | SUM=1000 | CARRY=0 | DONE=0
```

Figure: 13 ModelSim_Testbench

FPGA Implementation

Compilation verification:

Flow Status	Successful - Tue Jun 03 20:46:35 2025
Quartus Prime Version	18.1.0 Build 625 09/12/2018 SJ Lite Edition
Revision Name	L5_SerialAdder
Top-level Entity Name	serial_adder_top
Family	Cyclone IV E
Device	EP4CE115F29C7
Timing Models	Final
Total logic elements	21 / 114,480 (< 1 %)
Total registers	20
Total pins	17 / 529 (3 %)
Total virtual pins	0
Total memory bits	0 / 3,981,312 (0 %)
Embedded Multiplier 9-bit elements	0 / 532 (0 %)
Total PLLs	0 / 4 (0 %)

Figure: 14 Compilation Report

The pins consideration was assigned as following, in this case we add different elements for complete the functionality in the FPGA.

out carry_out	Output	PIN_G21
in clk	Input	PIN_Y2
out done	Output	PIN_F17
in in_a[3]	Input	PIN_Y23
in in_a[2]	Input	PIN_Y24
in in_a[1]	Input	PIN_AA22
in in_a[0]	Input	PIN_AA23
in in_b[3]	Input	PIN_AA24
in in_b[2]	Input	PIN_AB23
in in_b[1]	Input	PIN_AB24
in in_b[0]	Input	PIN_AC24

in rst	Input	PIN_AB28
in start	Input	PIN_M23
out sum[3]	Output	PIN_E24
out sum[2]	Output	PIN_E25
out sum[1]	Output	PIN_E22
out sum[0]	Output	PIN_E21

Figure: 15 PinPlanner

The RTL view can be observed as follows.

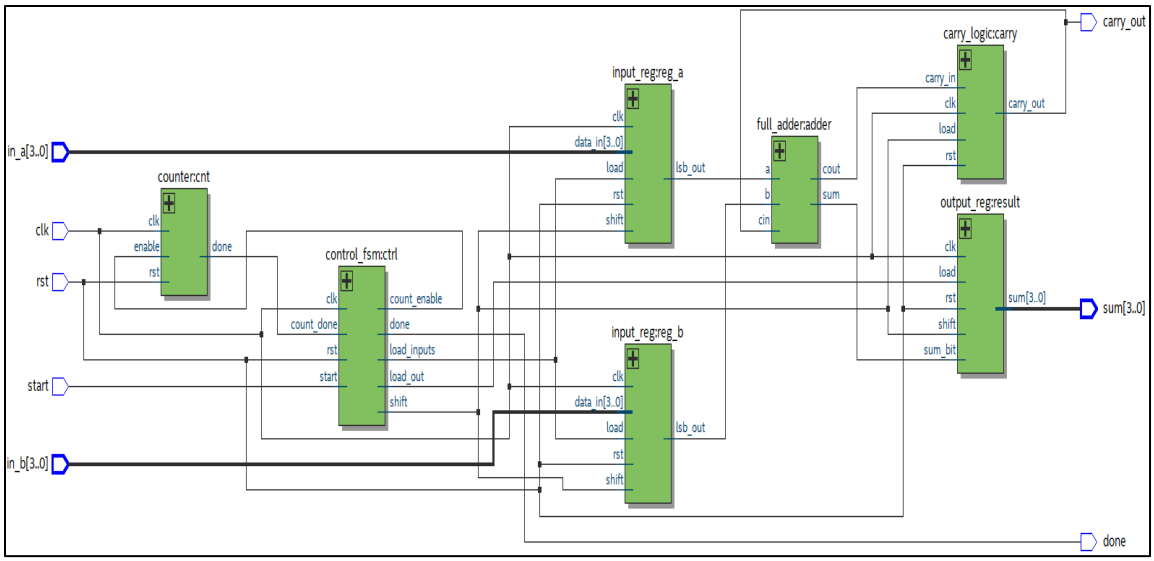


Figure: 16 RTL Viewer

FPGA Results

In this section, we performed some validation tests to ensure that the serial adder operations were carried out correctly and to verify its functionality on the board.

1.- A = 0011(3) , B= 0111(7) , SUM = 1010(10) → NO CARRY

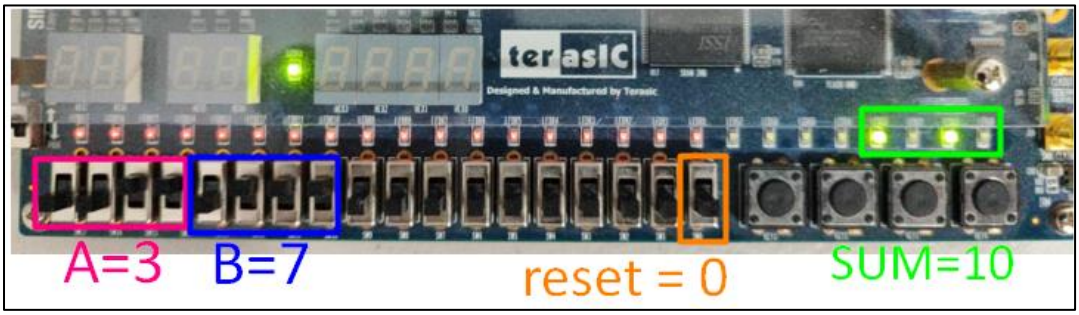


Figure: 17 Test1

1.- A = 0011(3) , B= 0111(7) , SUM = 1010(10) → NO CARRY

Figure: 18 Test2

GitHub Repository

https://github.com/CeicGitHub/Fundamentals_Of_Digital_Design_FPGA_3/tree/Lab5_SerialAdder

Conclusion / Issues

Cesar Eduado Inda Ceniceros

Personally, this practice represented a significant level of difficulty because I didn't know it was possible to work with the state machine part, and "FSM" is a topic we have recently been covering in this Module 4 in relation to traffic light control. In this regard, for example, using typedef enum helped me understand that many of these implementations are similar to C. Although not identical, it does represent a starting point for applying them. Additionally, working on the testbench using SystemVerilog was insightful, especially when dealing with certain keywords that the tool marked as incorrect. Although I still have some doubts about which keywords are exclusive to Verilog or SystemVerilog, I believe the understanding of the practice was adequately achieved.

Alonso Emmanuel Lopez Macias

I believe this practice took us more time to understand its scope because it didn't simply involve adding two numbers as in previous labs. We had to start implementing the use and behavior of state transitions, which was a challenge since we hadn’t worked with them in such depth before. I think we expanded a lot by creating many separate modules that we might have managed within a single file. However, due to doubts and uncertainties about whether everything would fail when integrating it all, we preferred to go the extra mile to ensure better functionality. I believe that, within the objective of the practice, what I take away is the understanding of many concepts that, even now in Module 4, are being reinforced through the labs from Module 3.